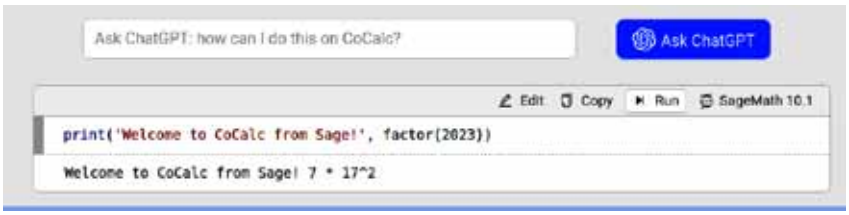

Figure 5: The CoCalc environment

However, I have found the unpaid version of CoCalc powerful enough for most requirements. Though not free (as in free beer), CoCalc is open source software hosted by SageMath Inc. The developer of CoCalc is also William Stein, the initial developer of SageMath. CoCalc is available online at '*https://cocalc.com/features/sage?utm_source=sagemath.org&utm_medium=icon*'. Additionally, the home page of SageMath, available at '*https://www.sagemath.org*', has a link to the CoCalc platform.

In the first few articles in this series, we will be using CoCalc, as new users can immediately start using SageMath without any installation. You don't even need to register to execute SageMath code with CoCalc. However, if you want to save your work for later, you need to register and log in. If you prefer, you can use one of your accounts on GitHub, Google, Facebook, X (Twitter), etc, to log into CoCalc. Figure 5 shows the CoCalc environment (with ChatGPT enabled) available on the home page itself. Notice that CoCalc offers SageMath version 10.01.

Now, log into the CoCalc platform to save your work for the future. Let us proceed to write the SageMath code to find the sum and average of an array of numbers. The code is:

```
Import numpy as np

arr = np.array([2, 4, 5, 4, 5, 5, 3, 6, 11, 7])
s = np.sum(arr)
avg = np.mean(arr)
print("Sum:", s)
print("Average:", avg)
```

The first line of code imports the Python library called NumPy. The second line uses the NumPy function *array( )* to create an array. The next two lines of code use the NumPy functions *sum( )* and *mean( )* to find the sum and average, respectively. The last two lines of code print these results. Upon execution in CoCalc, the above code will print the sum as 52 and the average as 5.2.

You may now feel tricked, as I have yet again provided Python code masquerading as SageMath code. I intentionally did this to highlight the crucial point that SageMath not only employs Python 3 syntax but can also seamlessly utilise Python libraries as if they are native to SageMath. This integration significantly enhances the capabilities of SageMath. However, I still believe that ending our discussion here may give the impression that I am just trying to sell a different version of Python to you. Trust me, that is not my intention. So, let us delve into the true strength of SageMath, which Python cannot replicate, through a straightforward example. Consider the pure SageMath code (not Python code) shown here:

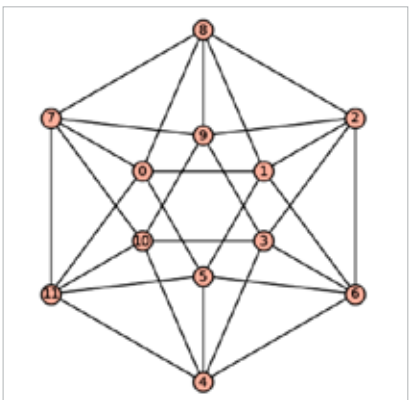

Figure 6: The icosahedral graph

```
g = graphs.IcosahedralGraph( )
g.show( )
```

First, let us attempt to understand the meaning of the code. In the first line of code, a variable *g* is assigned the value of an icosahedral graph created using the function *graphs.IcosahedralGraph( )* from SageMath's *graph* module. An icosahedral graph is a representation of the vertices and edges of an icosahedron, a Platonic solid. To learn more about icosahedrons and their properties, please refer to the Wikipedia article on Platonic solids. The second line of code calls the method *show( )* on the graph object *g*, which displays the icosahedral graph. Figure 6 illustrates the output of this code.

Now, it is time to conclude our discussion. In the next article in this series, we will discuss graphs, which are mathematical structures used to model pairwise relationships between various objects. But, why do we need to discuss such mathematical structures? In an age marked by widespread social media usage, graphs prove invaluable in capturing relationships between diverse entities such as people and objects. So we will delve into the fundamentals of graph theory—a branch of mathematics/computer science dedicated to studying graphs—and engage in substantial SageMath coding to process graphs. END

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He maintains a technical blog.